Автономная некоммерческая общеобразовательная организация
«Физтех-лицей» имени П. Л. Капицы

МСА - Многофункциональная система агентов ИИ.

Научный руководитель —
Пугин П. Ю.

Выполнили
ученики 9 «К» класса
Загатов Лев,
Ивашкин Сергей,
Картавцев Роман

г. Долгопрудный

1. Введение

1.1. Описание проекта

Многофункциональная система агентов ИИ (далее МСА) — программно-аппаратный комплекс, главной задачей которого является выполнение сложных интеллектуальных задач с использованием локальных больших языковых моделей (LLM). В отличии от стандартных чат-ботов, МСА реализует архитектуру мультиагентной оркестрации. Задача пользователя декомпозируется и выполняется каскадом специализированных агентов с использованием механизмов самопроверки, получения дополнительной точной актуальной информации из загруженных документов и сети „Интернет“ и работа с внешними инструментами по протоколу MCP.

Центральным инженерным достижением проекта является аппаратно-ориентированный инференс. Система спроектирована для работы на вычислительном кластере из видеокарт серии NVIDIA CMP, использовавшихся в прошлом для добычи криптовалют, конкретно CMP 50 HX и CMP 90 HX. Отличие этих ускорителей от классических заключается в ограничении пропускной способности шины PCI-E, а также, в случае моделей CMP 70 HX, CMP 90 HX и CMP 170 HX, ограничение на работу тензорных ядер обработки вычислений типа FP32/FP16. За счёт глубокого анализа набора команд и микроархитектурных ограничений этих ускорителей, кластер успешно выполняет инференс LLM.

1.1.1. Терминология и архитектурные компоненты системы

Инструмент агента ИИ — некоторая функция, которую может использовать агент ИИ.

Квантизация — целенаправленное уменьшение точности весов большой языковой модели, с целью экономии видеопамяти видеоускорителей. Бывает разных видов, в системе используется GGUF квантизация для обхода ограничений видеокарт CMP 90 HX в FP16 вычислениях. GGUF формат поддерживает хранение моделей в INT8 формате чего хватает для точности ответов, и что убирает FP16 нагрузку из инференса.

Протокол MCP — протокол, позволяющий агентам ИИ получать контекст из внешнего источника, и использовать в этом источнике предусмотренные функции. Функции для MCP протокола задаются на сервере, на котором и исполняются. [8]

Агент - модератор — агент ИИ, главной задачей которого является определить, какому специализированному агенту передать запрос дальше, а также определяет сам запрос этому агенту.

Агент - специалист — агент ИИ, который обрабатывает запрос, применяя все доступные ему инструменты, включая предоставленные по протоколу MCP. После чего возвращает ответ пользователю.

Система — Многофункциональная система агентов ИИ.

Система восприятия — компонент Системы, который отвечает за распознавание и хранение текстов, загруженных пользователем, а также за семантическую маршрутизацию запросов пользователей. В неё входят: векторная база данных Qdrant, система поиска в интернете SearXNG, OCR модель olmOCR-2-7b и 2 embedding модели: qwen3-embedding-0,6b и qwen3-embedding-8b, для семантической маршрутизации и обработки индексирования документов для загрузки в Qdrant соответственно, запущенные при помощи llama.cpp.

Система инференса — компонент Системы, который отвечает за инференс LLM, в неё входят: LLM qwen3-8b, запущенная через llama.cpp в GGUF квантизации Q8_0 (все веса модели урезаются до INT8 из FP16[5]), а также сервер, обеспечивающий взаимодействие клиента (веб-сайта), агента ИИ, LLM и системы восприятия.

1.2. Актуальность

Актуальность проекта обеспечена синергией двух проблем современной IT-индустрии:

1. Развёртывание современных LLM требует дорогостоящего серверного оборудования (уровня NVIDIA A100/H100), доступ к которому затруднён из-за высокой стоимости и санкционных ограничений. В то же время, после масштабного изменения алгоритмов добычи криптовалют, на рынке образовался избыток графических процессоров, использовавшихся для добычи криптовалют, в частности видеоускорители серии NVIDIA CMP, которые производитель ограничил в графической производительности, урезав конвейеры обработки вычислений данных типа FP32/FP16 (FP16 потому что отдельных конвейеров обработки FP16 у архитектуры NVIDIA Ampere, нет даже на стандартных видеоускорителях серии NVIDIA GeForce на соответствующей архитектуре, у архитектуры NVIDIA Turing другой случай), сделав их формально непригодными для использования в задачах обучения и дообучения LLM, графических задачах и иных задачах, которые не относятся к криптографии и добыче криптовалют. Данный проект доказывает возможность использования этого оборудования, снижая стоимость развёртывания локального ИИ в десятки раз.

2. Многофункциональные системы агентов ИИ сегодня создаются повсеместно, в силу того, что они лучше линейных алгоритмов инференса LLM во многих задачах, таких как решение проблем, вопросы по теме общества, психологии и другие[8]. Агентская составляющая системы в первую очередь позволяет большой языковой модели опираться на актуальную информацию, получаемую из инструментов агента, таких как поиск в интернете или поиск по сохранённым в

массиве документов для RAG документам, в отличии от изначальной LLM, которая может при генерации опираться лишь на данные, на которых она была обучена. Но в России нет образцов мультиагентных систем, находящихся в открытом доступе, во многих компаниях в России ведётся разработка агентских систем на основе существующих решений с открытым исходным кодом, или же на основе собственных решений. Но они предназначены только для решения задач компании и недоступны внешнему пользователю. Также, такие программы есть на зарубежном рынке, но в большинстве случаев они недоступны из региона РФ, имеют закрытый исходный код и не имеют возможности самостоятельного размещения.

1.3. Сравнение аналогов

1.3.1. Таблица

Сравнение аналогов представлено в таблице:

Таблица 1, сравнение аналогов

Параметр	Microsoft Copilot	Gemini	ChatGPT	OpenClaw	Гигачат	МСА
Наличие интеграции с другими сервисами по MCP	-	+	-	+	-	+
Доступ к исходному коду программы	-	-	-	+	-	+
Поиск источников информации в интернете	+	+	+	+	+	+
Работа с документами	+	+	+	+	+	+
Доступ из РФ	-	-	-	+	+	+
Возможность локального размещения	-	-	-	+	-	+

Под доступом из РФ подразумевается доступ с Российского IP из России, что невозможно с ChatGPT[1], Microsoft Copilot[2] и Google Gemini[3] по причине блокировки пользователей из РФ от создателей из-за наложенных санкций.

Во время создания проекта также появился проект OpenClaw (бывший ClawdBot), однако в нём сделана ставка на автономного агента со множеством разнонаправленных инструментов, и автономию этого агента, а не

последовательную цепочку различных агентов с основной целью сделать ответ на вопрос наилучшим.

1.4. Функционал

1.4.1. Основной функционал

1.4.1.1. Отправка запроса

Пользователь может отправить запрос системе через сайт. Для этого он:

1. Выбирает модель LLM, которая будет обрабатывать его запрос. Если модель не указана, она выбирается автоматически.
2. Вводит запрос в поле запроса. Если для генерации необходим контекст из документа, то прикрепляет его. Если для генерации необходим контекст из Интернета, нажимает специальную кнопку с надписью „Поиск“.
3. Нажимает кнопку отправить запрос.

Затем:

1. Программа запускается, запрос передаётся агенту-модератору.
2. Он определяет наиболее подходящего задаче агента-специалиста, при помощи поиска на основе алгоритма RAG и передаёт ему задачу.
3. Агент-специалист принимает задачу, обрабатывает её и возвращает ответ пользователю.

1.4.1.2. Система чатов

Пользователь может переключаться между разными своими запросами, так называемыми чатами. В пределах чата система сохраняет контекст диалога, однако не сохраняет его вне чатов.

1.4.1.3 Добавление внешних сервисов

В систему интегрирован протокол MCP, благодаря которому система может взаимодействовать с внешними сервисами. Для добавления сервиса в протокол отдельного пользователя, он может перейти в настройки аккаунта, в раздел „Личные сервисы“, где может указать всю необходимую информацию для использования MCP.

1.5. Алгоритмы и структуры данных

Инференс LLM (больших языковых моделей), происходящий через llama.cpp. Нужен как основа системы, использующая формат API совместимый с OpenAI API, для совместимости с другими сервисами, которые могут предоставлять доступ по совместимому API.

Сложность имплементации: низкая, так как механизм запуска нейросетей был создан отдельно.

RAG — Retrieval-Augmented Generation — Алгоритм хранения и использования данных для генерации ответов нейросетями с использованием проверенных источников. Алгоритм происходит в 2 этапа:

- Извлечение. Вход: Запрос пользователя. Процесс: запрос пользователя преобразуется в векторное представление при помощи модели кодирования запросов, этот вектор используется для поиска релевантной информации в базе знаний, из этой базы выбирается топ-К наиболее релевантных документов/фрагментов текста. Выход: набор релевантных документов или фрагментов текста(контекст), извлеченных из базы знаний.

- Генерация. Вход: запрос пользователя и извлеченный контекст. Процесс: запрос пользователя и извлеченный контекст объединяются в 1 входной текст, который передается в LLM, которая анализирует его и генерирует ответ. Выход: сгенерированный ответ на запрос пользователя, основанный на извлеченной информации из внешних источников.

Сложность имплементации: высокая, на данный момент существует множество различных подходов к созданию RAG, самые продвинутые требуют особенно глубоких познаний в сфере ИИ.

Процесс работы самой программы показан на рис. 1 в приложении А.

1.6. Научная новизна и оригинальные технические решения

В ходе работы над МСА были применены следующие инновационные подходы:

1. Асимметричная маршрутизация вычислений. Вычислительно-тяжёлая генерация текста размещена на вычислительном узле с 3x CMP 90 HX, основанных на архитектуре Ampere, с использованием целочисленной квантизации (Q8_0 GGUF), инференс LLM при которой происходит с использованием инструкции DP4A, что полностью нивелирует блокировку тензорных ядер. Операции необходимые для работы системы и чувствительные к типу данных вынесены на вычислительный узел с 3x CMP 50 HX, ограничения на FP32/FP16 на которых ограничены лишь аппаратно, что позволяет восстановить изначальную производительность чипа лишь заменой программного обеспечения, обеспечивающего работу чипа, также называемого драйвера.

2. Вставка ролевого запроса в конец. Вставка ролевой инструкции агента в конец пользовательского промпта вместо системного сообщения (system prompt). Используется в сочетании с функцией Context Reuse современных серверов инференса LLM (llama.cpp, версия 2026).

   Стандартный подход: [SYSTEM: Ты — эксперт по коду] + [USER: запрос] → Генерация → KV-cache инвалидируется при смене роли

Late-Binding подход: [USER: запрос] + [ROLE: Ты — эксперт по коду] → Генерация → KV-cache префикса сохраняется

3. Динамическая инъекция инструментов. Механизм работы:
   Стандартный подход:

   Агент "Coder" → инструменты: [калькулятор, поиск кода]
   Агент "RAG" → инструменты: [Qdrant search]
   Агент "Web" → инструменты: [поиск в интернете]
   Динамическая инъекция:
   Тема определена -> Агент получает базовую роль → Инструменты добавляются по условию:
   - Если запрос требует документации → +RAG
   - Если запрос требует актуальных данных → +Web Search
   - Если запрос требует вычислений → +Calculator

1.7. Цели и задачи

1.7.1. Общая цель

Разработка программно-аппаратного комплекса многофункциональной системы агентов ИИ (МСА), демонстрирующего возможность эффективного инференса больших языковых моделей на вычислительном оборудовании с ограниченными ресурсами (resource-constrained environments) за счёт аппаратно-ориентированной оптимизации и архитектурных решений уровня программного обеспечения.

1.7.2. Научные цели

1. Исследовать методы обхода аппаратного снижения производительности видеоускорителей серии NVIDIA CMP в FP вычислениях. Успехом можно считать подтверждение работоспособности инференса квантизированных методом GGUF больших языковых моделей на NVIDIA CMP 90 HX.

2. Разработка алгоритмов компенсации латентности ограниченной шины PCIe (x1). Успех: снижение латентности системы за счёт использования Cache reuse.

3. Исследование эффективности семантической маршрутизации на основе embedding. Успех: латентность роутинга 50-100 мс, при точности >85%.

4. Исследование возможности расширения контекста LLM в условиях ограниченной VRAM. Успех: Контекст 32K+ для модели 8B в 30 ГБ VRAM через KV-cache Q8_0, со сжатием истории диалога при превышении историей чата 20K токенов.

1.7.3. Инженерные цели

1. Сборка и тестирование вычислительного кластера на основе видеокарт серии NVIDIA CMP. Статус: выполнено.

2. Выяснение аппаратных ограничений процессора кластера (Intel Pentium G4400), а также сборка под него llama.cpp. Статус: выполнено.
3. Развёртывание единой базовой модели qwen3-8b (Q8_0) на системе инференса. Статус: выполнено.
4. Реализация семантического роутера на основе qwen3-embedding-0,6b + Qdrant. Статус: выполнено.
5. Интеграция протокола MCP для подключения внешних сервисов. Статус выполнено.
6. Интеграция веб-интерфейса. Статус: выполнено.
7. Написание документации. Статус: выполнено.

1.7.4. Ожидаемые результаты

1. Работающий прототип системы с веб-интерфейсом, доступный для локального развёртывания.

2. Доказательство концепции использования GPU серии NVIDIA CMP для инференса LLM.

3. Методология оптимизации инференса в условиях resource-constrained environments (Edge AI).

4. Публикация тезисов на конференции «Старт в Инновации».

5. Открытый исходный код проекта для сообщества.

1.8. Список литературы

1. Михаил. OpenAI заблокировала API-трафик ChatGPT из России: что это значит? [Электронный ресурс] — Режим доступа: https://external.software/archives/44254, свободный — (Дата обращения: 10.11.2025).
2. Yang Z. Copilot не работает в России.[Электронный ресурс] — Режим доступа: https://learn.microsoft.com/ru-ru/answers/questions/2353481/copilot, свободный — (Дата обращения: 10.11.2025).
3. Google. Gemini пока не поддерживается в вашей стране[Электронный ресурс] — Режим доступа: https://gemini.google.com/?hl=ru, свободный — (Дата обращения: 10.11.2025).
4. Georgi Georganov. Llama.cpp/README.md[Электронный ресурс] — Режим доступа: https://github.com/ggml-org/llama.cpp?tab=readme-ov-file, свободный — (Дата обращения: 10.11.2025).
5. Uygar Kurt. WHICH QUANTIZATION SHOULD I USE? A UNIFIED EVALUATION OF LLAMA.CPP QUANTIZATION ON LLAMA-3.1-8B-INSTRUCT — С. 1-17 —

электронная версия — Режим доступа: https://arxiv.org/pdf/2601.14277 , свободный — (Дата обращения: 01.03.2026).

6. Науменко А.О. Технология RAG(Retrieval-Augmented Generation) как инновационный подход в LLM — С. 1-10 — электронная версия — Режим доступа: https://cyberleninka.ru/article/n/tehnologiya-rag-retrieval-augmented-generation-kak-innovatsionnyy-podhod-v-llm/viewer, свободный — (Дата обращения: 10.11.2025).
7. Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, Xiangliang Zhang. Large Language Model based Multi-Agents: A Survey of Progress and Challenges — С. 1-15 — электронная версия — Режим доступа: https://arxiv.org/abs/2402.01680, свободный — (Дата обращения: 11.11.2025).
8. Неизвестный. What is Model Context Protocol (MCP)?[Электронный ресурс] — Режим доступа: https://modelcontextprotocol.io/docs/getting-started/intro, свободный — (Дата обращения: 11.11.2025).
9. Xubin Wang, Weijia Jia. Optimizing Edge AI: A Comprehensive Survey. Efficient Inference for Edge Large Language Models: A Survey. [Электронный ресурс] — Режим доступа: https://arxiv.org/html/2501.03265v1, свободный — (Дата обращения: 03.03.2026).
10. XING KANGWEI. Exploration of Cryptocurrency Mining-Specific GPUs in AI Applications. [Электронный ресурс] — Режим доступа: https://arxiv.org/abs/2505.03782, свободный — (Дата обращения: 03.03.2026).

2. Руководство пользователя

2.1. Посещение сайта

Взаимодействие с МСА осуществляется через веб-интерфейс. Система доступна в браузере по адресу сервера и порту 6190 (например, http://localhost:6190).

После загрузки страницы пользователю открывается рабочее пространство, включающее:

1. Боковую панель: список прошлых обращений (чатов). При выборе чата открывается история переписки с возможностью продолжить диалог.

2. Выбор модели: рядом со списком чатов расположено меню выбора большой языковой модели (LLM).

3. Поле ввода запроса: основная область для написания команд.

4. Панель инструментов (в поле ввода): кнопки для загрузки документов, активации поиска в Интернете, включения специфических функций больших языковых моделей (LLM) и перехода в настройки MCP.

2.2. Взаимодействие с МСА

Взаимодействие с системой происходит после того, как пользователь вводит свой запрос к системе в поле ввода запроса, для отправки запроса необходимо нажать кнопку „Отправить“ или клавишу „Enter“. Через некоторое время после отправки запроса, под ним появится ответ системы, в зависимости от запроса, данные для ответа могут быть предоставлены из:

1. Загруженных пользователем документов.

2. Электронной сети „Интернет“.

3. Внешних ресурсов, подключенных через протокол MCP.

2.3. Настройка внешних ресурсов MCP

В разделе настроек MCP пользователь может добавить URL-адрес нового сервера инструментов. Это позволит агентам-специалистам выполнять специфические задачи, такие как работа с базами данных или интеграция с календарем.

3. Руководство программиста

3.1. Системные требования

Для развёртывания системы МСА требуется следующая аппаратная и программная конфигурация:

3.1.1. Аппаратные требования

Компонент	Минимальные требования	Рекомендуемые требования
Хост (Система Инференса)	Intel Pentium G4400, 8 ГБ RAM	Intel Core i3+, 16 ГБ RAM
Хост (Система Восприятия)	Intel Pentium G4400, 8 ГБ RAM	Intel Core i3+, 16 ГБ RAM
GPU (Система Инференса)	3x NVIDIA CMP 90 HX 10 GB	3x NVIDIA CMP 90 HX 10 GB
GPU (Система Восприятия)	3x NVIDIA CMP 50 HX 10 GB	3x NVIDIA CMP 50 HX 10 GB
Интерконнект	1x PCI-E 1.0	4x PCI-E 1.0 (больше невозможно)
Накопитель	200 ГБ SATA 3.0 HDD	500 ГБ NVME SSD
Сеть	1 гбит/с между узлами, 100 мбит/с доступ в интернет	10 гбит/с между узлами, 1 гбит/с доступ в интернет

3.1.2. Программные требования

Компонент	Версия	Примечание
ОС	Ubuntu 22.04 / Debian 12	Рекомендуется Linux
Python	3.10+	Для backend компонентов
Docker	24.0+	Опционально, для контейнеризации
NVIDIA Driver (Система Инференса)	535.x+	Подойдёт стандартный драйвер
NVIDIA Driver (Система Восприятия)	580.x+ patched	Патч находится по следующему адресу https://github.com/dartraiden/NVIDIA-patcher
NVIDIA Container Toolkit	1.13+	Только для Docker запуска
Llama.cpp	Позже 2025 года релиза	Инференс всех моделей, для работы на старых процессорах требуются

		аргументы компиляции, указанные ниже, а также аргументы для отключения Tensor cores в инференсе[10].
Qdrant	1.12+	Векторная база данных
SearXNG	1.1.0+	Поисковый агрегатор

Примечание: флаги компиляции llama.cpp, на примере Intel Pentium G4400:

-DCMAKE_C_FLAGS="-march=x86-64 -mno-avx -mno-avx2 -mno-fma -mno-f16c -mno-bmi -mno-bmi2 -mno-avx512f" -DCMAKE_CXX_FLAGS="-march=x86-64 -mno-avx -mno-avx2 -mno-fma -mno-f16c -mno-bmi -mno-bmi2 -mno-avx512f" -DGGML_CUDA=ON -DCMAKE_CUDA_ARCHITECTURES="86;75" -DGGML_CUDA_NO_TENSOR_CORES=ON -DCUDAToolkit_STATIC=ON -DGGML_NATIVE=OFF -DGGML_AVX=OFF DGGML_AVX2=OFF -DGGML_AVX512=OFF -DGGML_FMA=OFF -DGGML_F16C=OFF -DGGML_CPU_ALL_VARIANTS=.OFF

3.2. Пошаговая инструкция развёртывания с использованием Docker

Шаг 1: Подготовка репозитория

```
git clone https://github.com/KEKweb/System-MCA.git && cd System-MCA
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Шаг 2: Настройка переменных окружения

2 варианта:

1. Экспорт переменных окружения

```
export PARAMETER=“VALUE“
```

2. Файл .env Образец этого файла представлен в Приложении Б, части 1.

Шаг 3: Настройка Docker Compose

Образец конфигурационного файла docker-compose.yml представлен в Приложении Б, части 2.

Шаг 4: Создание Dockerfile

Необходимо создать Dockerfile на Node A и Node B, для того, чтобы сборка llama.cpp точно поддерживалась конкретным процессором. Образцы Dockerfile файлов приведены в Приложении Б, частях 3 и 4. В случае, когда процессор поддерживает стандартную сборку llama.cpp этот шаг не нужен, но необходимо изменить docker-compose.yml, чтобы он использовал стандартный Docker image llama.cpp.

Шаг 5: Запуск системы

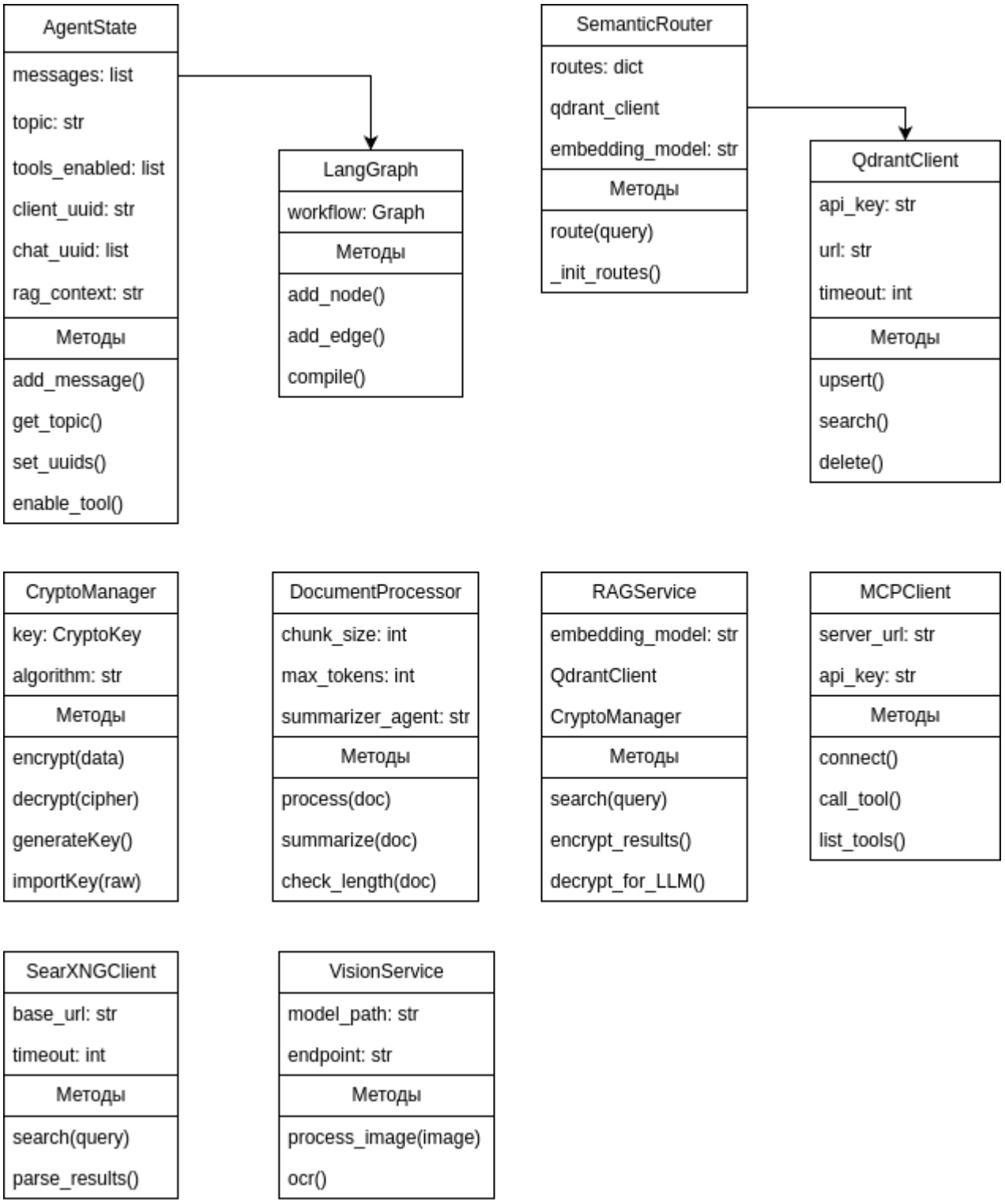
```
# Запуск всех сервисов
docker-compose up -d

# Проверка статуса
docker-compose ps

# Просмотр логов
docker-compose logs -f msa-backend
```

Шаг 6: Проверка работоспособности

```
# Проверка Qdrant
curl http://localhost:6333/readyz

# Проверка llama-server Node A
curl http://localhost:11434/health

# Проверка llama-server Модели embedding
curl http://localhost:11435/health

# Проверка llama-server Модели OCR
curl http://localhost:11436/health

# Проверка llama-server Модели router
curl http://localhost:11437/health

# Проверка SearXNG
curl http://localhost:8080/healthz

# Проверка системы
curl http://localhost:6190/health
```

3.3. Описание структур данных программы

3.3.1. Диаграммы классов



3.3.2. Описание классов

Класс	Назначение	Ключевые методы
AgentState	Хранение состояния графа агентов	add_message(), get_topic(), enable_tool(), set_uuids()

SemanticRouter	Семантическая классификация запросов	route(query), _init_routes()
QdrantClient	Взаимодействие с векторной БД	upsert(), search(), delete()
CryptoManager	Шифрование/расшифровка данных	encrypt(), decrypt(), generateKey()
DocumentProcessor	Обработка документов (суммаризация)	process(), summarize(), check_length()
RAGService	Поиск и управление RAG	search(), encrypt_results(), decrypt_for_llm()
MCPClient	Подключение внешних сервисов	connect(), call_tool(), list_tools()
SearXNGClient	Поиск в интернете через SearXNG	search(), parse_results()
VisionService	Обработка изображений (OCR, VL)	process_image(), ocr()
LangGraph	Оркестрация агентов (State Machine)	add_node(), add_edge(), compile()

3.4. Описание схемы используемой БД

3.4.1. Структура именования коллекций Qdrant

Тип коллекции	Формат имени	TTL	Назначение
Временная RAG	{client_uuid}_{chat_uuid}	24 часа	Обработка запросов сессии
Routes	routes	Бессрочно	Семантическая маршрутизация

Примеры:

- f47ac10b-58cc-4372-a567-0e02b2c3d479_chat_a1b2c3d4 (временная)
- routes (общая)

3.4.2. Структура точки данных

Поле	Тип	Описание	Шифрование
id	UUID	Уникальный идентификатор фрагмента	Нет
vector	float[1024]	Векторное представление текста	Нет (embedding не шифруется)
payload.encrypted_content	base64	Зашифрованный текст фрагмента	AES-256-GCM
payload.iv	base64	Initialization Vector для AES	Нет
payload.tag	base64	Authentication Tag для AES	Нет
payload.metadata.filename	base64	Имя исходного файла	AES-256-GCM

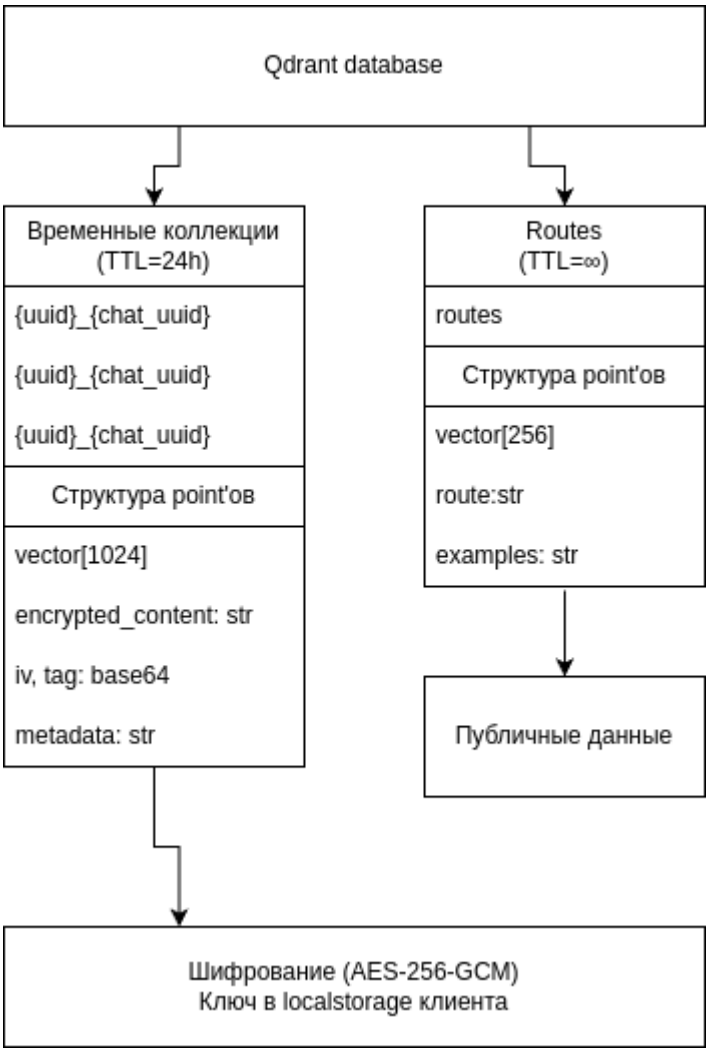
payload.metadata.chunk_index	int	Индекс чанка в документе	Нет
payload.metadata.created_at	timestamp	Дата создания	Нет
payload.metadata.token_count	int	Количество токенов	Нет

3.4.3. Словесное описание БД

Векторная база данных Qdrant используется для двух целей:

1. Коллекции пользователей ({client_uuid}_{chat_uuid}):
   - Хранят зашифрованные фрагменты документов пользователя
   - Каждая коллекция изолирована от других
   - Автоматически удаляются через 24 часа (TTL)
   - Ключ шифрования хранится только на клиенте
2. Коллекция "routes" (общая для всех):
   - Хранит эталонные примеры для семантической маршрутизации
   - Не шифруется (публичные данные)
   - Доступна всем узлам системы
   - Бессрочное хранение

3.4.4. Схема БД



3.5. Описание протоколов взаимодействия программы

3.5.1. Протокол MCP (Model Context Protocol)

Параметр	Значение
Назначение	Подключение внешних сервисов к агентам ИИ
Формат	JSON-RPC 2.0
Порт	Настраивается пользователем (по умолчанию 8080)
Аутентификация	API-key в заголовке запроса
Транспорт	HTTP/WebSocket

Пример запроса MCP:

```
{
  "jsonrpc": "2.0",
  "id": 1,
```

```
  "method": "tools/call",
  "params": {
    "name": "search_database",
    "arguments": {
      "query": "пользовательские данные"
    }
  }
}
```

3.5.2. SSE (Server-Sent Events)

Параметр	Значение
Назначение	Стриминг токенов ответа в реальном времени
Endpoint	/api/stream
Content-Type	text/event-stream
Метод	POST

Пример события SSE:

```
event: token
 {"token": "Привет"}

event: done
 {"finish_reason": "stop"}
```

3.5.3. OpenAI-compatible API

Параметр	Значение
Назначение	Совместимость с llama-server и другими инференс-серверами
Endpoint	/v1/chat/completions
Метод	POST
Порт Node A	11434
Порт Node B (Embedding)	11435
Порт Node B (Vision)	11436

3.5.4. HTTP/API между узлами

Endpoint	Метод	Описание
/api/embed	POST	Векторизация текста (Node B, порт 11435)
/api/route	POST	Семантическая маршрутизация (Node B)
/api/generate	POST	Генерация текста (Node A, порт 11434)
/api/vision	POST	Обработка изображений (Node B, порт 11436)

/api/summarize	POST	Суммаризация документа (Node A)

3.5.5. SearXNG API

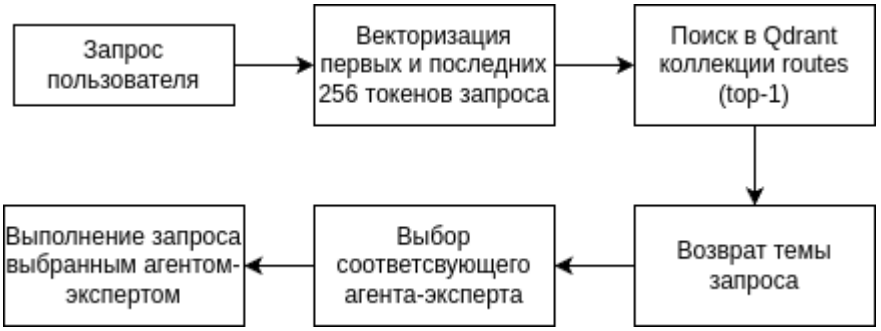
Параметр	Значение
Назначение	Поиск в интернете
Endpoint	/search
Метод	GET/POST
Порт	8080
Формат ответа	JSON

Пример запроса:

```
curl "http://searxng:8080/search?q=запрос&format=json"
```

3.6. Описание алгоритмов системы

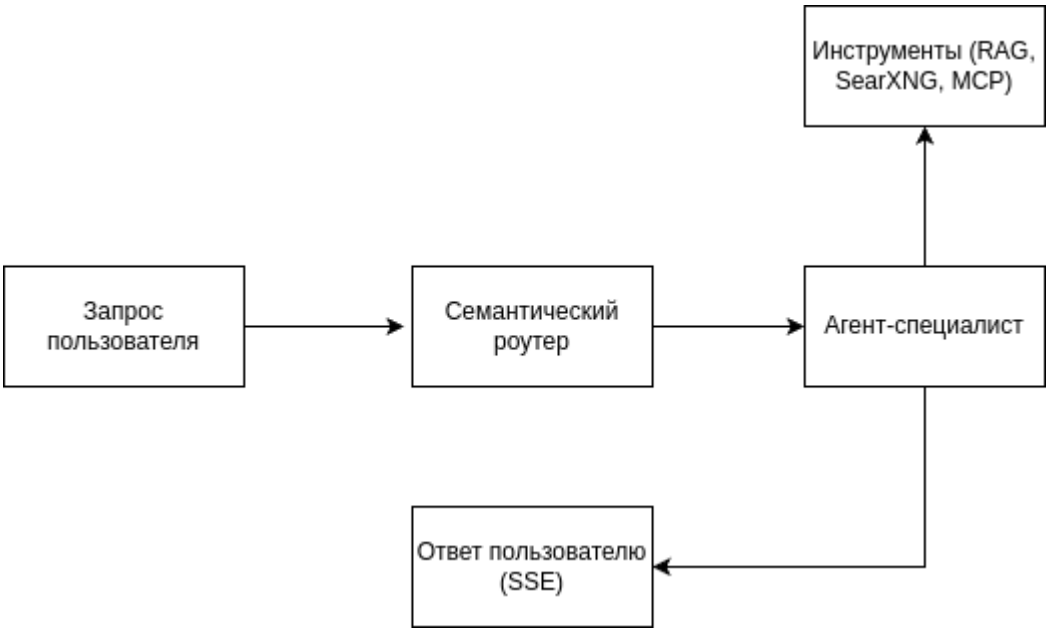
3.6.1. Блок-схема семантической маршрутизации



Словесное описание:

1. Запрос пользователя векторизуется через embedding-модель (qwen3-embedding-0,6b)
2. Осуществляется поиск ближайшего эталонного примера в коллекции "routes" (Qdrant)
3. Тема найденного примера определяется как целевая для маршрутизации
4. Запрос передаётся агенту-исполнителю на Node A с определённой темой

3.6.2. Блок-схема обработки запроса



Словесное описание:

1. Семантический роутер определяет тему запроса
2. Агент-исполнитель получает запрос с определённой темой
3. При необходимости агент вызывает инструменты (RAG, SearXNG, MCP)
4. Генерация ответа через llama-server с Late-Binding Role Injection
5. Ответ возвращается пользователю через SSE (стриминг токенов)

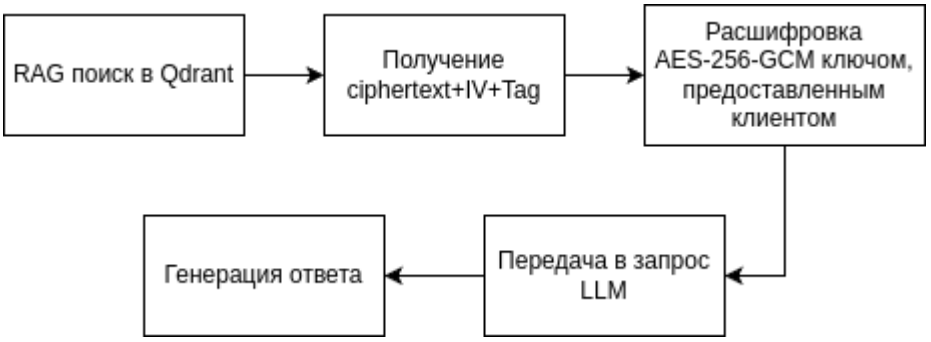
3.6.3. Блок-схема обработки документа

Словесное описание:

1. Пользователь загружает документ через веб-интерфейс
2. Проверка длины документа (лимит: 10K токенов на документ)
3. При превышении: вызов агента-суммаризатора (Qwen-3-8B, non-thinking mode)
4. Суммаризация: 10K токенов → ~3K токенов с сохранением семантики

5. Разбиение на чанки по 512 токенов
6. Шифрование каждого чанка (AES-256-GCM, ключ клиента из localStorage)
7. Векторизация открытого текста (до шифрования) на Node B
8. Сохранение в Qdrant (vector + encrypted payload)
9. Коллекция автоматически удаляется через 24 часа (TTL)

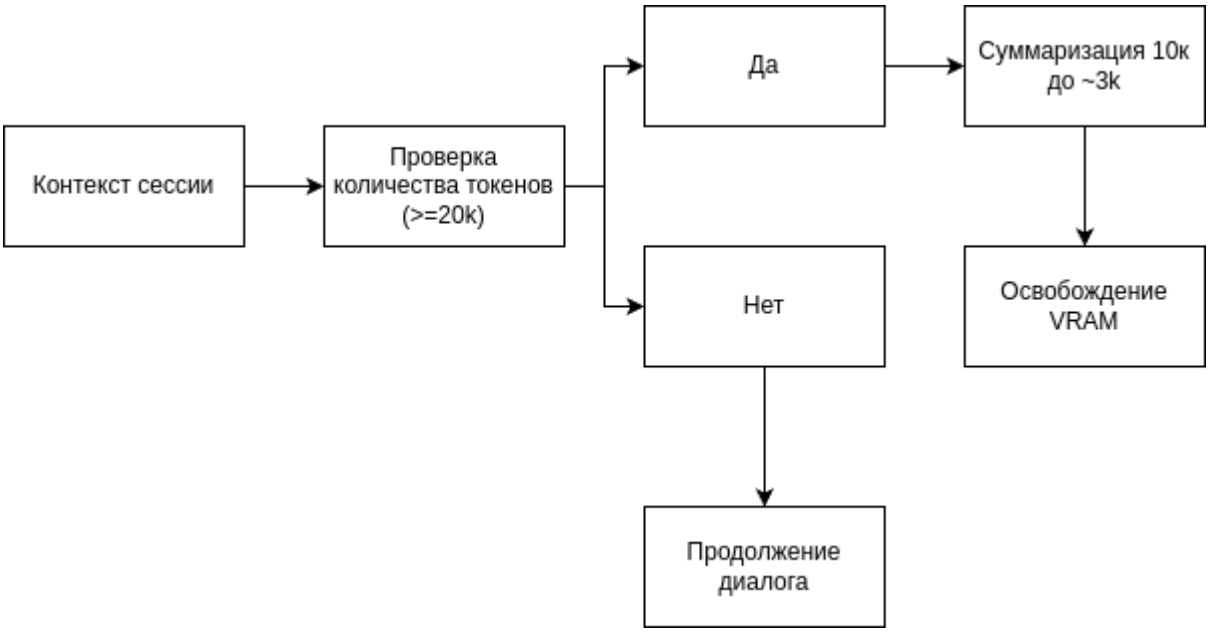
3.6.4. Блок-схема расшифровки для инференса



Словесное описание:

1. RAG поиск возвращает зашифрованные чанки из Qdrant
2. Клиент получает ciphertext + IV + tag
3. Расшифровка AES-256-GCM на клиенте (ключ из localStorage)
4. Открытый текст передаётся в промпт модели
5. Модель генерирует ответ на основе расшифрованного контекста
6. Ответ стримится пользователю через SSE

3.6.5. Управление контекстом



Словесное описание:

1. При каждом запросе проверяется текущая длина контекста сессии
2. Если контекст ≥ 20K токенов: вызов агента-суммаризатора
3. Первые 10K токенов сжимаются в summary (~1K токенов)

4. Контекст уменьшается до ~11K токенов
5. Освобождается ~2-3 ГБ VRAM на сессию
6. Максимальный контекст: 30K токенов (YaRN + Q8_0 KV-cache)

3.7. Основные переменные окружения/параметры конфигурации

Переменная	Описание	Значение по умолчанию
BASE_URL	Адрес llama-server генерация	http://localhost:11434
EMBEDDING_URL	Адрес llama-server embedding	http://localhost:11435
VISION_URL	Адрес llama-server vision	http://localhost:11436
ROUTER_URL	Адрес llama-server router	http://localhost:11437
SEARXNG_URL	Адрес поискового агрегатора SearXNG	http://localhost:8080
DATABASE_URL	Адрес векторной базы данных Qdrant	http://localhost:6333`
LOG_LEVEL	Уровень логирования	INFO`
SOD_DOCKER	Флаг запуска в Docker	true (из env)
LLM_MODEL_PATH	Путь к базовой модели	/models/qwen3-8b-instruct-q8_0.gguf
EMBEDDING_MODEL_PATH	Путь к embedding модели	/models/qwen3-embedding-4b-fp16.gguf
VISION_MODEL_PATH	Путь к vision модели	/models/qwen3-vl-8b-q8_0.gguf
ROUTER_MODEL_PATH	Путь к router модели	/models/qwen3-embedding-0,6b-fp16.gguf
ROUTER_TOP_K	Количество результатов поиска в Qdrant	1
KV_CACHE_QUANTIZATION	Тип квантования KV-cache	q8_0
CONTEXT_LENGTH	Максимальная длина контекста	30720
SUMMARIZATION_THRESHOLD	Порог суммаризации контекста	20480
DOCUMENT_MAX_TOKENS	Лимит токенов на документ	10240
RAG_TTL_SECONDS	TTL временных коллекций RAG	86400 (24 часа)

CLIENT_ENCRYPTION	Включить клиентское шифрование	true

3.8. Персистентность данных и TTL

3.8.1. TTL 24 часа — архитектурная особенность

Все временные коллекции RAG имеют установленное время жизни (TTL) 24 часа. Это архитектурное решение, а не ошибка.

Обоснование:

Причина	Объяснение
Безопасность	Данные не хранятся бессрочно на сервере — снижение риска утечки
152-ФЗ	Автоматическое удаление персональных данных после завершения обработки
Экономия ресурсов	Критично для оборудования с ограниченной VRAM/RAM

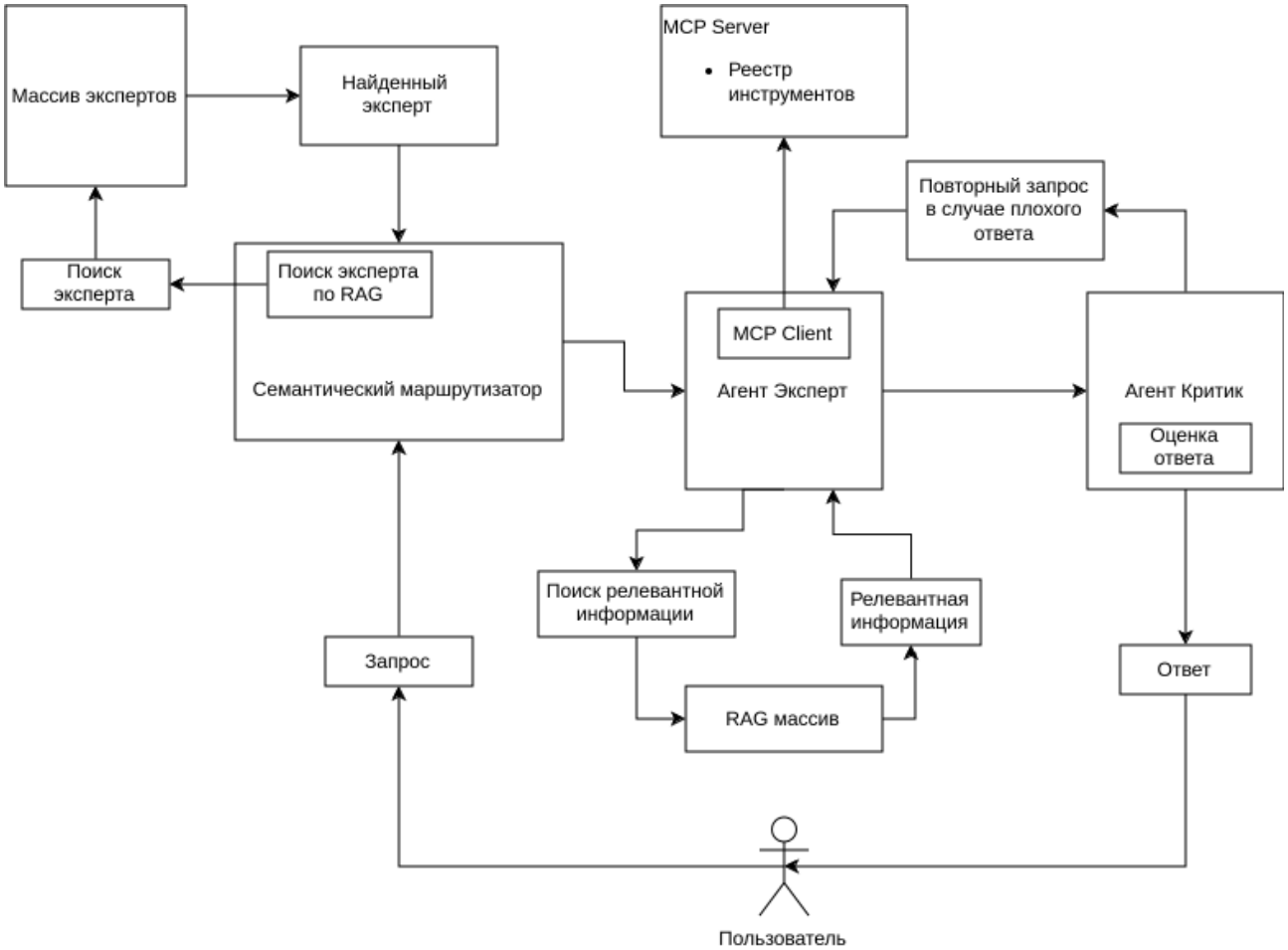
3.9. Выводы

Программный комплекс МСА представляет собой модульную распределённую систему с чётким разделением ответственности между компонентами и повышенными требованиями к безопасности данных:

1. Оркестрация: LangGraph обеспечивает управление состоянием и маршрутизацию между агентами с учётом client_uuid и chat_uuid.
2. Инференс: llama-server предоставляет OpenAI-compatible API для генерации текста на узле Node A (3× CMP 90HX) с использованием INT8-квантования (DP4A).
3. Восприятие: Узел Node B (3× CMP 50HX) отвечает за embedding, семантическую маршрутизацию, vision и RAG.
4. Хранение: Qdrant обеспечивает векторный поиск с клиентским шифрованием (AES-256-GCM) и TTL 24 часа для временных коллекций.
5. Безопасность: Все пользовательские данные шифруются на клиенте перед отправкой в Qdrant. Ключ шифрования никогда не передаётся на сервер. TTL обеспечивает автоматическое удаление данных через 24 часа.
6. Суммаризация: Документы >10K токенов и контекст сессии >20K токенов автоматически сжимаются агентом-суммаризатором для предотвращения перегрузки VRAM.
7. Интеграция: MCP позволяет подключать внешние сервисы без изменения кода агентов. SearXNG обеспечивает поиск в интернете без отслеживания.

8. Масштабируемость: Архитектура системы допускает горизонтальное масштабирование (добавление узлов инференса) и вертикальное масштабирование (увеличение контекста, количества параллельных клиентов) при сохранении безопасности данных.

Приложение А. Архитектура Программы



Запрос пользователя отправляется через сайт, после чего обрабатывается так, как показано на схеме.

Приложение Б. Листинги

Б1. Образец файла .env

```
# .env — MCA Stack Configuration
# Этот файл используется обоими docker-compose (node-a и node-b).

# --- URLs сервисов ---
# NODE A (локально на Node A, внешний IP для Node B)
DATABASE_URL=http://qdrant:6333
BASE_URL=http://llama-server-node-a:11434
SEARXNG_URL=http://searxng:8080
```

```
# NODE B (внешний IP Node B — указать реальный адрес)
EMBEDDING_URL=http://<NODE_B_IP>:11435
VISION_URL=http://<NODE_B_IP>:11436
ROUTER_URL=http://<NODE_B_IP>:11437

# --- Пути к моделям (внутри контейнера, смонтировано из ./models) ---
LLM_MODEL_PATH=/models/qwen3-8b-instruct-q8_0.gguf
EMBEDDING_MODEL_PATH=/models/qwen3-embedding-8b-fp16.gguf
VISION_MODEL_PATH=/models/qwen3-vl-8b-q8_0.gguf
ROUTER_MODEL_PATH=/models/qwen3-embedding-0.6b-fp16.gguf

# --- Параметры инференса (Node A — LLM) ---
# ctx=30720 — намеренное ограничение для максимизации числа
параллельных слотов.
# При KV Q8_0: ~9 параллельных пользователей по 30k токенов в 30GB VRAM.
# При KV Q4_0: ~19 параллельных пользователей (меньше точность KV).
CONTEXT_LENGTH=30720

# Допустимые значения: f16, q8_0, q4_0
# q8_0: баланс качество/размер, рекомендуется для production
# q4_0: максимальный concurrency (~19 слотов), небольшая деградация
точности
KV_CACHE_QUANTIZATION=q8_0

# Число параллельных слотов — должно соответствовать
KV_CACHE_QUANTIZATION:
#   q8_0 → PARALLEL_SLOTS=9
#   q4_0 → PARALLEL_SLOTS=19
#   f16  → PARALLEL_SLOTS=4
PARALLEL_SLOTS=9

GPU_LAYERS=99
BATCH_SIZE=512
UBATCH_SIZE=64

# --- Параметры инференса (Node B — Perception) ---
EMBEDDING_CTX_LENGTH=8192
VISION_CTX_LENGTH=8192
# ctx=512 хватает для роутинга (первые+последние 256 токенов запроса)
ROUTER_CTX_LENGTH=512

# --- Управление контекстом ---
SUMMARIZATION_THRESHOLD=20480
```

```
DOCUMENT_MAX_TOKENS=10240
RAG_TTL_SECONDS=86400

CLIENT_ENCRYPTION=true
LOG_LEVEL=INFO
SOD_DOCKER=true
```

Б2. Образец файла docker-compose.yml

```
version: '3.8'

# docker-compose.yml — MCA Stack (единый файл для обоих узлов)
# Запуск на Node A:
#   docker compose --profile node-a up -d
# Запуск на Node B:
#   docker compose --profile node-b up -d
# Одинаковая копия файла лежит на обеих машинах.
# Переменные читаются из .env в той же директории.

services:

  # NODE A — Generation Node (3x CMP 90HX, sm_86, DP4A)

  qdrant:
    profiles: [node-a]
    image: qdrant/qdrant:latest
    container_name: qdrant
    ports:
      - "6333:6333"
      - "6334:6334"
    volumes:
      - qdrant_storage:/qdrant/storage
      - qdrant_snapshots:/qdrant/snapshots
    environment:
      - QDRANT__SERVICE__GRPC_PORT=6334
    restart: unless-stopped

  searxng:
    profiles: [node-a]
    image: searxng/searxng:latest
    container_name: searxng
    ports:
      - "8080:8080"
    volumes:
      - ./searxng:/etc/searxng
```

```
    environment:
      - SEARXNG_BASE_URL=http://localhost:8080
    restart: unless-stopped

  llama-server-node-a:
    profiles: [node-a]
    build:
      context: .
      dockerfile: Dockerfile.node-a
      args:
        LLAMA_CPP_REF: master
    container_name: llama-server-node-a
    ports:
      - "11434:11434"
    volumes:
      - ./models:/models
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              device_ids: ["0", "1", "2"]
              capabilities: [gpu]
    command: >
      --model ${LLM_MODEL_PATH:-/models/qwen3-8b-instruct-q8_0.gguf}
      --host 0.0.0.0
      --port 11434
      --ctx-size ${CONTEXT_LENGTH:-30720}
      -ngl ${GPU_LAYERS:-99}
      --batch-size ${BATCH_SIZE:-512}
      --ubatch-size ${UBATCH_SIZE:-64}
      --cache-type-k ${KV_CACHE_QUANTIZATION:-q8_0}
      --cache-type-v ${KV_CACHE_QUANTIZATION:-q8_0}
      --parallel ${PARALLEL_SLOTS:-9}
      --split-mode layer
      --flash-attn
    restart: unless-stopped

  msa-backend:
    profiles: [node-a]
    build:
      context: .
      dockerfile: Dockerfile.backend
    container_name: msa-backend
```

```
    ports:
      - "6190:8080"
    volumes:
      - ./files:/home/user/files
    environment:
      - DATABASE_URL=http://qdrant:6333
      - BASE_URL=http://llama-server-node-a:11434
      - EMBEDDING_URL=${EMBEDDING_URL}
      - VISION_URL=${VISION_URL}
      - ROUTER_URL=${ROUTER_URL}
      - SEARXNG_URL=http://searxng:8080
      - SOD_DOCKER=true
      - LOG_LEVEL=${LOG_LEVEL:-INFO}
      - LLM_MODEL_PATH=${LLM_MODEL_PATH}
      - CONTEXT_LENGTH=${CONTEXT_LENGTH:-30720}
      -
SUMMARIZATION_THRESHOLD=${SUMMARIZATION_THRESHOLD:-20480}
      - DOCUMENT_MAX_TOKENS=${DOCUMENT_MAX_TOKENS:-10240}
      - RAG_TTL_SECONDS=${RAG_TTL_SECONDS:-86400}
      - CLIENT_ENCRYPTION=${CLIENT_ENCRYPTION:-true}
      - KV_CACHE_QUANTIZATION=${KV_CACHE_QUANTIZATION:-q8_0}
      - PARALLEL_SLOTS=${PARALLEL_SLOTS:-9}
    depends_on:
      - qdrant
      - searxng
      - llama-server-node-a
    restart: unless-stopped

  # NODE B — Perception & RAG Node (3x CMP 50HX, sm_75, FP16)

  llama-server-embedding:
    profiles: [node-b]
    build:
      context: .
      dockerfile: Dockerfile.node-b
      args:
        LLAMA_CPP_REF: master
    container_name: llama-server-embedding
    ports:
      - "11435:11434"
    volumes:
      - ./models:/models
    deploy:
      resources:
```

```
            reservations:
              devices:
                - driver: nvidia
                  device_ids: ["0"]
                  capabilities: [gpu]
        command: >
          --model
${EMBEDDING_MODEL_PATH:-/models/qwen3-embedding-8b-fp16.gguf}
          --host 0.0.0.0
          --port 11434
          --ctx-size ${EMBEDDING_CTX_LENGTH:-8192}
          -ngl ${GPU_LAYERS:-99}
          --embedding
          --batch-size 512
          --ubatch-size 64
        restart: unless-stopped

      llama-server-vision:
        profiles: [node-b]
        build:
          context: .
          dockerfile: Dockerfile.node-b
          args:
            LLAMA_CPP_REF: master
        container_name: llama-server-vision
        ports:
          - "11436:11434"
        volumes:
          - ./models:/models
        deploy:
          resources:
            reservations:
              devices:
                - driver: nvidia
                  device_ids: ["1"]
                  capabilities: [gpu]
        command: >
          --model ${VISION_MODEL_PATH:-/models/qwen3-vl-8b-q8_0.gguf}
          --host 0.0.0.0
          --port 11434
          --ctx-size ${VISION_CTX_LENGTH:-8192}
          -ngl ${GPU_LAYERS:-99}
          --cache-type-k ${KV_CACHE_QUANTIZATION:-q8_0}
          --cache-type-v ${KV_CACHE_QUANTIZATION:-q8_0}
```

```
      --batch-size 512
      --ubatch-size 64
    restart: unless-stopped

  llama-server-router:
    profiles: [node-b]
    build:
      context: .
      dockerfile: Dockerfile.node-b
      args:
        LLAMA_CPP_REF: master
    container_name: llama-server-router
    ports:
      - "11437:11434"
    volumes:
      - ./models:/models
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              device_ids: ["2"]
              capabilities: [gpu]
    command: >
      --model
${ROUTER_MODEL_PATH:-/models/qwen3-embedding-0.6b-fp16.gguf}
      --host 0.0.0.0
      --port 11434
      --ctx-size ${ROUTER_CTX_LENGTH:-512}
      -ngl ${GPU_LAYERS:-99}
      --embedding
      --batch-size 64
      --ubatch-size 32
    restart: unless-stopped

volumes:
  qdrant_storage:
  qdrant_snapshots:
```

Б3. Образец Dockerfile, универсальный для Node A и Node B

```
# --- Stage 1: Build ---
FROM nvidia/cuda:12.2.2-devel-ubuntu22.04 AS builder

ENV DEBIAN_FRONTEND=noninteractive
```

```
RUN apt-get update && apt-get install -y \
   git \
   cmake \
   build-essential \
   libcurl4-openssl-dev \
   ca-certificates \
   && rm -rf /var/lib/apt/lists/*

ARG LLAMA_CPP_REF=master

WORKDIR /build
RUN git clone https://github.com/ggml-org/llama.cpp.git . \
   && git checkout ${LLAMA_CPP_REF}

# ---------------------------------------------------------------------------
# CMAKE FLAGS — каждый флаг критичен, не убирать без понимания
последствий
#
# CPU-side (host C++):
#   -march=x86-64          baseline ISA, совместим с G4400
#   -mno-avx/-avx2/-avx512 G4400 не имеет AVX → SIGILL без этих флагов
#   -mno-fma               нет FMA3 на G4400 (SKD052)
#   -mno-f16c              нет F16C на G4400
#   -mno-bmi/-bmi2         SKD052: CPUID врёт про BMI, реально не работает
#
# CUDA-side (device kernels):
#   CUDA_ARCHITECTURES=86;75  компилируем только под sm_86 и sm_75
(CMP 90HX и CMP 50HX соответсвенно)
#   NO_TENSOR_CORES=ON     ключевой хак: убирает mma.sync путь,
#                          форсирует DP4A (dp4a инструкция INT32 ALU),
#                          который на CMP 90HX не заблокирован
#   CUDAToolkit_STATIC=ON  статическая линковка cudart → нет зависимости
#                          от версии runtime в финальном образе
#
# GGML CPU-side:
#   GGML_NATIVE=OFF        не автодетектить возможности хоста (важно!)
#   GGML_AVX/AVX2/etc=OFF  явно отключить, дублирует -mno-* для cmake
# ---------------------------------------------------------------------------
RUN cmake -B build \
   -DCMAKE_BUILD_TYPE=Release \
   -DCMAKE_C_FLAGS="-march=x86-64 -mno-avx -mno-avx2 -mno-fma
-mno-f16c -mno-bmi -mno-bmi2 -mno-avx512f" \
   -DCMAKE_CXX_FLAGS="-march=x86-64 -mno-avx -mno-avx2 -mno-fma
-mno-f16c -mno-bmi -mno-bmi2 -mno-avx512f" \
```

```
    -DGGML_CUDA=ON \
    -DCMAKE_CUDA_ARCHITECTURES="86;75" \
    -DGGML_CUDA_NO_TENSOR_CORES=ON \
    -DCUDAToolkit_STATIC=ON \
    -DGGML_NATIVE=OFF \
    -DGGML_AVX=OFF \
    -DGGML_AVX2=OFF \
    -DGGML_AVX512=OFF \
    -DGGML_FMA=OFF \
    -DGGML_F16C=OFF \
    -DGGML_CPU_ALL_VARIANTS=OFF \
    -DLLAMA_CURL=ON

RUN cmake --build build --config Release -j$(nproc) --target llama-server

RUN objdump -d build/bin/llama-server | grep -E "ymm|zmm|vfmadd" && \
    echo "WARNING: AVX/FMA instructions found in binary!" && exit 1 || \
    echo "OK: No forbidden instructions detected."

FROM nvidia/cuda:12.2.2-runtime-ubuntu22.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    libcurl4 \
    ca-certificates \
    && rm -rf /var/lib/apt/lists/*

COPY --from=builder /build/build/bin/llama-server /usr/local/bin/llama-server

VOLUME ["/models"]

EXPOSE 11434

ENTRYPOINT ["llama-server"]
```